

CSIE 5452, Spring 2026: Homework 1

B13902022 Yu-Chi,Lai

1 Timing Analysis of the CAN Protocol: Part I (12pt)

1.1

Firstly, set $Q_0 = B_0 = C_1 = 30$, since no other task has higher priority than μ_0 , the terms in the summation is zero.

Hence, the worst-case response time $R_0 = Q_0 + C_0 = 30 + 10 = 40$ (msec).

Iteration	LHS (Q_0)	B_0	RHS	Stop?
1	30	30	30	yes

1.2

Let $Q_1 = B_1 = C_1 = 30$ as the beginnning.

Iteration	LHS (Q_1)	B_1	j	$Q_1 + \tau$	T_j	$\lceil \frac{Q_1 + \tau}{T_j} \rceil$	C_j	RHS	Stop?
1	30	30	0	30.1	50	1	10	40	no
2	40	30	0	40.1	50	1	10	40	yes

Thus, the worst-case response time of μ_1 is $R_1 = Q_1 + C_1 = 40 + 30 = 70$ (msec).

1.3

Let $Q_2 = B_2 = C_2 = 20$ as the beginnning. After 3 iterations, Q_2 converges. Hence, the worst case response

Iteration	LHS (Q_2)	$\lceil \frac{Q_2 + \tau}{T_0} \rceil C_0 + \lceil \frac{Q_2 + \tau}{T_1} \rceil C_1$	RHS	Stop?
1	20	40	60	no
2	60	50	70	no
3	70	50	70	yes

time $R_2 = Q_2 + C_2 = 70 + 20 = 90$ (msec).

2 Timing Analysis of the CAN Protocol: Part II (36pt)

2.1

$R_0 = 1$
 $R_1 = 2$
 $R_2 = 2$
 $R_3 = 3$
 $R_4 = 3$
 $R_5 = 4$
 $R_6 = 5$
 $R_7 = 8$
 $R_8 = 9$
 $R_9 = 9$
 $R_{10} = 10$
 $R_{11} = 19$
 $R_{12} = 19$
 $R_{13} = 20$
 $R_{14} = 29$
 $R_{15} = 29$
 $R_{16} = 30$

2.2

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  signed main() {
5      ifstream fin("input.dat");
6      int n;
7      double tau;
8      fin >> n;
9      fin >> tau;
10     vector<int> priority(n);
11     vector<double> transmission_time(n), period(n);
12     for (int i = 0; i < n; ++i) {
13         fin >> priority[i] >> transmission_time[i] >> period[i];
14     }
15     fin.close();
16
17     vector<int> res(n);
18     for (int i = 0; i < n; i++) {
19         double block = 0;
20         for (int j = 0; j < n; j++) {
21             if (priority[j] >= priority[i]) {
22                 block = max(block, transmission_time[j]);
23             }
24         }
25
26         double wait = block;
27         while (1) {

```

```

28     double result = block;
29     for (int j = 0; j < n; j++) {
30         if (priority[j] < priority[i]) {
31             result += transmission_time[j] * ceil((wait + tau) / period[j]);
32         }
33     }
34     if (result > period[i]) {
35         res[i] = -1;
36         break;
37     }
38     if (result - wait <= 1e-9 && result - wait >= -(1e-9)) {
39         res[i] = transmission_time[i] + wait;
40         break;
41     } else {
42         wait = result;
43         continue;
44     }
45 }
46 }
47
48 for (int i = 0; i < n; i++) {
49     cout << res[i] << endl;
50 }
51 }

```

3 Timing Analysis of Preemptive Fixed-Priority Scheduling (16pt)

3.1

Iteration	LHS (R_0)	C_0	RHS	Stop?
1	10	10	10	yes

3.2

Iteration	LHS (R_1)	C_1	j	R_1	T_j	$\frac{R_1}{T_j}$	C_j	RHS	Stop?
1	30	30	0	30	50	1	10	40	no
2	40	30	0	40	50	1	10	40	yes

3.3

Iteration	LHS (R_2)	C_2	j	R_2	T_j	$\frac{R_2}{T_j}$	C_j	RHS	Stop?
1	20	20	0	20	50	1	10	60	no
			1		200	1	30		
2	60	20	0	60	50	2	10	70	no
			1		200	1	30		
3	70	20	0	70	50	2	10	70	yes
			1		200	1	30		

3.4

Although preemptive fixed-priority scheduling is often disadvantageous to the lowest-priority task because higher-priority tasks can interrupt it multiple times, in this case τ_2 still has a smaller worst-case response time than μ_2 .

The key reason is that non-preemptive CAN analysis includes an additional blocking term B_2 :

$$Q_2 = B_2 + \sum_{j:P_j < P_2} \left\lceil \frac{Q_2 + \tau}{T_j} \right\rceil C_j, \quad R_{\mu_2} = Q_2 + C_2.$$

In contrast, preemptive fixed-priority analysis does not have this blocking term:

$$R_{\tau_2} = C_2 + \sum_{j:P_j < P_2} \left\lceil \frac{R_{\tau_2}}{T_j} \right\rceil C_j.$$

From the computed results, $R_{\mu_2} = 90$ msec while $R_{\tau_2} = 70$ msec. Therefore, for this task set, the reduction from removing non-preemptive blocking is larger than the extra delay caused by preemptions, so τ_2 ends up with a smaller worst-case response time.

4 Timing Analysis of TDMA-Based Protocols (12pt)**4.1**

$$(4, 10, 1, 2, 6, 7)$$

4.2

$$(4, 10, 0, 3, 5, 6, 10, 13, 15, 16)$$

4.3

$$(4, 10, 1, 2, 6, 7, 11, 12, 16, 17)$$

4.4

k	$\max_{1 \leq j \leq n} (s_{j+k} - s_j)$	$\min_{1 \leq i \leq m} (a_{i+k-1} - a_i)$	(Column-2) - (Column-3)
1	4	0	4
2	5	1	4
3	9	3	6
4	10	6	4

4.5

$$6 + 1 = 7$$

The worst case response time is 7.

5 MILP Linearization (12pt)**5.1****5.2****5.3****6 Signal Packing (12pt)****6.1****6.2****6.3**